

Cell 16:-

This part is the **SPARQL query template used to find neighbouring nodes around the seed nodes.**

Think of it as:

"Given these seed nodes, find all relationships connected to them, in either direction, and return the relationship details."

Let's go line by line.

1. Creating the query template

NEIGHBOUR_QUERY_TMPL = PREFIXES + ""

Here we create a variable called:

NEIGHBOUR_QUERY_TMPL

It stores a **SPARQL query**.

PREFIXES contains the namespace definitions needed by the query, such as:

rdf:

rdfs:

ex:

So this:

PREFIXES + ""

means:

Add the required RDF prefixes before the actual SPARQL query.

2. SELECT — What information do we want?

SELECT ?src ?srcLabel ?pred ?predLabel ?tgt ?tgtLabel

 ?description ?strength ?linkType

This tells SPARQL:

"Give me these pieces of information from the graph."

There are three main things:

Source

?src

The node where the relationship starts.

Example:

Aircraft

Predicate

?pred

The relationship between the nodes.

Example:

contains

Target

?tgt

The node where the relationship ends.

Example:

Engine

So the basic structure is:

Source — Predicate —> Target

Aircraft — contains —> Engine

3. Labels

?srcLabel

?predLabel

?tgtLabel

These are human-readable names.

For example, the URI might be:

<http://example.com/node/123>

but its label could be:

Aircraft

Similarly:

?pred → URI of relationship

?predLabel → "contains"

So labels make the result easier to understand.

4. Description

?description

This contains a description of the relationship.

For example:

"The aircraft contains the engine."

This can later provide useful evidence for retrieval.

5. Strength

?strength

This represents how strong or reliable the relationship is.

For example:

strength = 90

Later, this can be used for ranking.

6. Link type

?linkType

This tells us what type of link/relationship it is.

For example:

semantic

reference

dependency

depending on how the graph was constructed.

7. WHERE

WHERE {

This starts the actual graph-matching conditions.

In simple English:

"Now tell me what conditions the graph must satisfy."

8. VALUES ?seed

VALUES ?seed { %SEEDS% }

This is very important.

?seed represents the **starting node(s)** for traversal.

%SEEDS% is a placeholder.

Later Python replaces it with actual nodes.

For example:

%SEEDS%

might become:

<http://kg/Aircraft>

<http://kg/Engine>

So the query becomes something like:

VALUES ?seed {

 <http://kg/Aircraft>

 <http://kg/Engine>

}

Meaning:

"Start searching around these nodes."

9. Find RDF statements

?stmt a rdf:Statement ;

This says:

Find something called ?stmt that is an RDF Statement.

In this project, relationships are stored as **reified RDF statements**.

Instead of simply storing:

Aircraft → contains → Engine

the graph stores a statement node that contains information about the relationship.

Conceptually:

Statement

|
├— subject → Aircraft
├— predicate → contains
├— object → Engine
├— description → ...
├— strength → 90
└— linkType → ...

This is important because the relationship itself has extra information like:

- description
- strength
- link type

10. Get the source node

`rdf:subject ?src ;`

This says:

Get the subject/source of the statement and store it in ?src.

Example:

`?src = Aircraft`

11. Get the relationship

`rdf:predicate ?pred ;`

This gets the relationship/predicate.

Example:

?pred = contains

12. Get the target node

rdf:object ?tgt ;

This gets the target/object.

Example:

?tgt = Engine

So these three lines reconstruct the basic RDF triple:

?src — ?pred —> ?tgt

For example:

Aircraft — contains —> Engine

13. Get relationship description

ex:description ?description ;

This retrieves the description stored on the statement.

Example:

"The aircraft contains the engine."

14. Get relationship strength

ex:strength ?strength ;

This retrieves the relationship's strength.

Example:

90

15. Get link type

ex:linkType ?linkType .

This retrieves the type of relationship.

Example:

semantic

The . means:

This statement pattern is complete.

16. The MOST important line

```
FILTER(?src = ?seed || ?tgt = ?seed)
```

This is the key part of this query.

It says:

Find relationships where the seed node is either the source OR the target.

`||` means **OR**.

So it checks:

```
?src = ?seed
```

OR

```
?tgt = ?seed
```

Why do we need both directions?

Suppose the graph contains:

Document A — contains —> Component B

If the user asks about:

Component B

then:

Component B

is the **target**.

If we only checked:

```
?src = ?seed
```

we wouldn't find the relationship.

But this code checks:

`?src = ?seed OR ?tgt = ?seed`

So it finds:

Document A — contains —> Component B

Therefore:

The traversal can move in both directions around a seed node.

This is one of the most important reasons this query exists.

17. Get source label

`OPTIONAL { ?src rdfs:label ?srcLabel }`

Try to get the human-readable label of the source.

For example:

`?src = http://kg/node123`

`?srcLabel = "Aircraft"`

OPTIONAL means:

If a label exists, return it. If it doesn't exist, don't fail the entire query.

18. Get target label

`OPTIONAL { ?tgt rdfs:label ?tgtLabel }`

Same idea for the target.

Example:

`?tgt = http://kg/node456`

`?tgtLabel = "Engine"`

19. Get predicate label

`OPTIONAL { ?pred rdfs:label ?predLabel }`

This gets a readable name for the relationship.

For example:

?pred = http://example.com/contains

?predLabel = "contains"

Again, OPTIONAL means the query still works if the label is missing.

20. End of query

}

""

This closes the WHERE block and finishes the query template.

Simple example

Suppose our seed is:

Aircraft

And the graph contains:

Aircraft — contains —> Engine

Engine — connected_to —> Fuel System

Control System — controls —> Aircraft

The query checks both sides of the seed.

It can find:

Aircraft — contains —> Engine

because:

?src = Aircraft

And it can also find:

Control System — controls —> Aircraft

because:

?tgt = Aircraft

So the result can be:

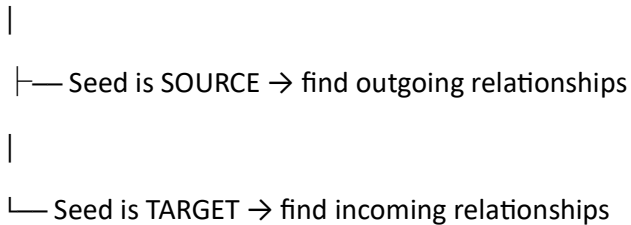
Source	Predicate	Target
Aircraft	contains	Engine
Control System	controls	Aircraft

In one sentence

NEIGHBOUR_QUERY_TMPL is a SPARQL template that takes seed KG nodes and finds every reified relationship connected to those nodes, whether the seed is the source or target, while also returning labels, description, strength, and link type.

The most important logic to remember:

Seed node



That is why this query is the foundation of the `traverse()` function in Cell 16.

It takes a list of document nodes and retrieves their actual document content from the Knowledge Graph.

This is different from the previous `NEIGHBOUR_QUERY_TMPL`, which finds **relationships**.

1. Create the query template

`DOC_CONTENT_TMPL = PREFIXES + ""`

We create a variable:

`DOC_CONTENT_TMPL`

It stores a SPARQL query.

`PREFIXES` adds the RDF namespace definitions required by the query.

So:

`PREFIXES + SPARQL query`

means:

"Prepare a complete SPARQL query with the required namespaces."

2. SELECT

SELECT ?doc ?label ?content

We are asking SPARQL to return **three things**:

?doc

The document node.

Example:

doc123

?label

The document's human-readable name.

Example:

Hydraulic Control System

?content

The actual text/content of the document.

Example:

The hydraulic control system consists of...

So the result is basically:

Document URI



Document Label



Document Content

3. WHERE

WHERE {

This starts the conditions for finding the documents.

4. VALUES ?doc { %DOCS% }

VALUES ?doc { %DOCS% }

This is very important.

%DOCS% is a **placeholder**.

Later, Python replaces it with the actual document nodes.

For example, suppose traversal finds:

doc101

doc205

doc310

The query could become:

```
VALUES ?doc {  
    <doc101>  
    <doc205>  
    <doc310>  
}
```

Meaning:

"Only retrieve content for these specific documents."

This is important because we don't want to retrieve the content of every document in the entire KG.

5. Find the document content

?doc ex:documentContent ?content .

This is the main part of this query.

It says:

For each selected document, find its ex:documentContent.

Conceptually, your KG contains something like:

Document123

|

| documentContent



"The hydraulic system consists of..."

So:

?doc



ex:documentContent



?content

The actual text gets stored in:

?content

6. Get the document label

OPTIONAL { ?doc rdfs:label ?label }

This says:

If the document has a label, get it.

For example:

?doc = doc123

?label = "Hydraulic Control System"

OPTIONAL is important because a document might not have a label.

If there is no label, the query **still returns the document and its content.**

7. Why do we need this query in Cell 16?

This is the important part.

The previous query:

NEIGHBOUR_QUERY_TMPL

finds relationships:

Document A

↓ contains

Component B

But finding the relationship isn't enough.

Eventually, the RAG system needs **actual text** that can be given to the next stage/LLM.

So we have:

Question

↓

Cell 15

Extract entities

↓

Cell 16

Find KG nodes

↓

Traverse relationships

↓

Identify useful documents

↓

DOC_CONTENT_TMPL

↓

Retrieve actual document text

Simple example

Suppose traversal finds these documents:

doc101

doc205

Then this query retrieves:

doc	label	content
-----	-------	---------

doc101	Engine System	"The engine system consists of..."
--------	---------------	------------------------------------

doc205	Fuel System	"The fuel system provides..."
--------	-------------	-------------------------------

Now Cell 17 or the later retrieval stage can use this content as **evidence/context**.

The key difference from NEIGHBOUR_QUERY_TMPL

NEIGHBOUR_QUERY_TMPL

Answers:

"What is connected to this node?"

Example:

Engine

↓ connected_to

Fuel System

DOC_CONTENT_TMPL

Answers:

"What is the actual text inside this document?"

Example:

Fuel System

↓

"The fuel system provides fuel to the engine..."

So remember it like this:

NEIGHBOUR_QUERY_TMPL

↓

Find relationships

↓

Find useful document nodes

↓

DOC_CONTENT_TMPL



Get actual document text

In one sentence: DOC_CONTENT_TMPL converts the **document nodes discovered during graph traversal into actual text content**, so the retrieved graph evidence can ultimately be used to answer the user's question.

Find KG nodes whose labels contain the entity phrase extracted from the question.

1. Select node and label

```
SELECT DISTINCT ?node ?label
```

Gets the **node URI** and its **human-readable label**.

2. Get nodes having labels

```
?node rdfs:label ?label .
```

Example:

Node → "Hydraulic Control System"

3. Match the phrase

```
FILTER(CONTAINS(LCASE(STR(?label)), %PHRASE%))
```

Checks whether the label contains the search phrase, ignoring case.

Example:

Phrase: "hydraulic control"

Label: "Hydraulic Control System"



MATCH

4. Ignore predicate nodes

```
FILTER(!STRSTARTS(STR(?node), "%PREDMARK%"))
```

This excludes nodes that represent **predicates/relationships**.

So it focuses on actual **entity/document nodes**.

5. Maximum 10 results

LIMIT 10

Returns at most **10 matching nodes**.

In one line:

SUBSTRING_TMPL searches the KG for up to 10 entity nodes whose labels contain the extracted phrase, while ignoring predicate nodes.

_STOP — Simple Explanation

This is a set of **common words that are not useful for entity matching**.

`_STOP = {"the", "a", "an", "of", "for", ...}`

For example, in:

"What is the relationship of the engine with the fuel system?"

It removes words like:

what, is, the, of, with

and keeps the meaningful words:

relationship, engine, fuel, system

Why?

Because common words don't help identify KG entities.

So `_STOP` helps the **lexical matching/fallback** focus on meaningful words and improves matching accuracy.

These are **three small helper functions** used by Cell 16.

1. _tokens(s)

`def _tokens(s):`

```
    return {t for t in re.findall(r"[a-z0-9]+", (s or "").lower())
           if t not in _STOP and len(t) > 2}
```

Purpose: Break text into useful words.

Example:

"What is the hydraulic control system?"

becomes roughly:

{"hydraulic", "control", "system"}

It:

- converts to lowercase
- extracts words
- removes _STOP words
- removes words with 2 or fewer characters

Used for: lexical fallback matching.

2. `_values_block(uris)`

`def _values_block(uris):`

`return " ".join(f"<{u}>" for u in uris if "<" not in u and ">" not in u)`

Purpose: Convert a list of KG node URIs into a format that SPARQL VALUES can use.

Example:

`["http://kg/A", "http://kg/B"]`

becomes:

`<http://kg/A> <http://kg/B>`

Used for: giving the traversal query its current seed/frontier nodes.

3. `_sparql_literal(s)`

`def _sparql_literal(s):`

`return "'" + s.replace("\\", "\\").replace("'", "\\'") + "'"`

Purpose: Safely convert normal Python text into a SPARQL string.

For example, if the text contains:

`"`

or:

\

it escapes those characters so the SPARQL query doesn't break.

In short:

Function	Purpose
<code>_tokens()</code>	Converts text → useful words
<code>_values_block()</code>	Converts URIs → SPARQL VALUES format
<code>_sparql_literal()</code>	Makes text safe to put inside SPARQL

These are just **helper functions** that make the main retrieval functions easier and safer.

link_entities() — Simple Explanation

The main purpose of this function is:

Take the phrases extracted from the user's question and find the matching nodes in the Knowledge Graph.

It uses **two methods**:

1. SPARQL substring matching → **precision**
2. Fuzzy matching → **recall**

1. Start the function

```
def link_entities(phrases):
```

phrases contains the entity phrases extracted by Cell 15.

Example:

```
"hydraulic control system"
```

```
"fuel pump"
```

2. Create best

```
best = {}
```

This stores the **best matching KG node and its score**.

3. Process every phrase

for phrase, _cls in phrases:

It takes each extracted phrase one by one.

_cls is the entity class, but it isn't used here.

4. First method — SPARQL substring matching

q = (SUBSTRING_TMPL

.replace("%PHRASE%", _sparql_literal(phrase.lower()))

.replace("%PREDMARK%", PREDICATE_MARKER))

It prepares the SPARQL query from SUBSTRING_TMPL.

For example:

Phrase:

"hydraulic control"

It searches KG labels for something containing:

hydraulic control

5. Execute the SPARQL query

for row in g.query(q):

This searches the Knowledge Graph.

If it finds:

Hydraulic Control System

then that node is considered a match.

6. Give substring match a score of 95

if best.get(node, (0,))[0] < 95:

best[node] = (95.0, phrase)

A substring match gets:

95 score

So it is considered a strong match.

7. Second method — Fuzzy matching

```
for label_lc, score in fuzzy_match(  
    phrase.lower(),  
    ALL_LABELS,  
    MAX_SEEDS_PER_PHRASE,  
    MATCH_CUTOFF  
):
```

Now it uses **fuzzy matching** to find labels that are similar even if they aren't exact substring matches.

Example:

Query:

"hydraulic control"

KG label:

"Hydraulic Control System"

Fuzzy matching can give something like:

96.2

8. Keep the best score

if node not in best or score > best[node][0]:

```
    best[node] = (score, phrase)
```

If the same node is found multiple times, it keeps the **highest score**.

So:

Node A → 85

Node A → 94

Node A → 91

Final:

Node A → 94

9. Return the results

```
return [  
    (n, NODE_LABEL.get(n, n), s, p)  
    for n, (s, p) in sorted(  
        best.items(),  
        key=lambda kv: kv[1][0],  
        reverse=True  
    )  
]
```

Finally, it returns:

node

label

score

original phrase

and sorts them from **highest score to lowest score**.

Example:

```
[  
    (node1, "Hydraulic Control System", 97, "hydraulic control"),  
    (node2, "Hydraulic System", 89, "hydraulic control")  
]
```

In one sentence:

link_entities() takes Cell 15's extracted phrases, searches the KG using exact-ish substring matching and fuzzy matching, keeps the best matches, and returns ranked KG nodes to be used as traversal seeds.

lexical_fallback() — Simple Explanation

The purpose of this function is:

If `link_entities()` cannot find good entities, this function tries to find KG nodes using simple word matching.

No LLM, embeddings, or vector search.

1. Take the question

```
def lexical_fallback(question, limit=5):
```

It receives the user's full question.

limit=5 means it will return at most **5 matching nodes**.

2. Convert question into useful words

```
q_tokens = _tokens(question)
```

For example:

"What is the hydraulic control system?"

becomes:

```
{"hydraulic", "control", "system"}
```

because `_tokens()` removes common words like what, is, the.

3. If there are no useful words

```
if not q_tokens:
```

```
    return []
```

If nothing useful is found, return an empty result.

4. Check every KG label

```
for label_lc, nodes in LABEL_TO_NODES.items():
```

It goes through the KG's stored labels.

For example:

Hydraulic Control System

Fuel System

Engine

Pressure Sensor

5. Find common words

`overlap = q_tokens & _tokens(label_lc)`

This finds words that exist in **both**:

Question words

$\cap$

KG label words

Example:

Question:

`{"hydraulic", "control", "system"}`

KG label:

`{"hydraulic", "control", "system"}`

Overlap:

`{"hydraulic", "control", "system"}`

6. Calculate a score

`score = 100.0 * len(overlap) / max(len(_tokens(label_lc)), 1)`

It calculates how much of the KG label matches the question.

Example:

KG label:

`"Hydraulic Control System"`

3 matching words / 3 label words

= 100%

If only 2 of 3 words match:

$2 / 3 \times 100 = 66.7$

7. Store the matching node

```
scored.append(  
    (n, NODE_LABEL.get(n, n), min(score, 99.0), "lexical")  
)
```

It stores:

node

label

score

"lexical"

`min(score, 99.0)` ensures the fallback score never becomes 100.

8. Sort by score

```
scored.sort(key=lambda x: x[2], reverse=True)
```

Highest matching score comes first.

9. Return top 5

```
return scored[:limit]
```

Since `limit=5`, it returns the **top 5 matching KG nodes**.

Simple example

Question:

"How does the hydraulic control system work?"

Fallback may find:

1. Hydraulic Control System → 99

2. Hydraulic System → 66

3. Control System → 66

4. Hydraulic Pump → 33

In one sentence:

lexical_fallback() is a backup method that compares the important words in the question with KG node labels, scores their overlap, and returns the top matching nodes when normal entity linking fails.

his is the **main graph traversal function** of Cell 16.

Its job is:

Start from the seed nodes and repeatedly find their neighbouring nodes using SPARQL, up to a certain number of hops.

I'll go **line by line**, but keep it simple.

1. Define the function

def traverse(seeds):

The function receives seeds.

These are the KG nodes identified from the user's question.

Example:

Engine

Fuel System

These become the starting points.

2. Create tracking variables

edges, seen, visited = [], set(), {n: (0, s) for n, _l, s, _p in seeds}

This creates 3 things.

edges

edges = []

Stores all the relationships we discover.

Example:

Engine → connected_to → Fuel System

seen

seen = set()

Keeps track of relationships we have already seen.

This prevents duplicate edges.

visited

visited = {n: (0, s) for n, _l, s, _p in seeds}

Keeps track of nodes that have already been discovered.

For each seed, it stores:

hop = 0

seed score = original score

So if:

Engine → score 95

it stores something like:

Engine → (0, 95)

3. Create the first frontier

frontier = [n for n, _l, _s, _p in seeds]

frontier contains the nodes we need to explore **right now**.

Initially, it contains the seed nodes.

Example:

frontier = [Engine, Fuel System]

4. Start hop-by-hop traversal

for hop in range(1, MAX_HOPS + 1):

This says:

Repeat the graph search from hop 1 up to MAX_HOPS.

For example, if:

MAX_HOPS = 3

then:

Hop 1

Hop 2

Hop 3

5. Stop if there is nothing to explore

if not frontier or len(edges) >= MAX_EDGES:

 break

Stop if:

Condition 1

not frontier

There are no more nodes to explore.

OR

Condition 2

len(edges) >= MAX_EDGES

We've already collected the maximum allowed number of edges.

This prevents excessive traversal.

6. Convert frontier to SPARQL format

block = _values_block(frontier[:MAX_FRONTIER])

frontier contains Python strings/URIs.

_values_block() converts them into SPARQL VALUES format.

For example:

```
frontier = [Engine, FuelSystem]
```

becomes something like:

```
<Engine> <FuelSystem>
```

MAX_FRONTIER limits how many nodes we send into one query.

7. Stop if conversion failed

if not block:

```
    break
```

If there are no valid nodes in the SPARQL block, stop traversal.

8. Run the SPARQL query

try:

```
    rows = list(
        g.query(
            NEIGHBOUR_QUERY_TMPL.replace("%SEEDS%", block)
        )
    )
```

This is the main retrieval step.

It takes the earlier query:

```
NEIGHBOUR_QUERY_TMPL
```

and replaces:

```
%SEEDS%
```

with the current frontier nodes.

Then:

```
g.query(...)
```

runs the query against the Knowledge Graph.

The result is stored in:

rows

So conceptually:

Current frontier



SPARQL query



Neighbour relationships



rows

9. Handle query errors

except Exception as e:

```
    print(f" hop {hop} query failed: {e}")
```

```
    break
```

If the SPARQL query fails:

- print the error
- stop the traversal

10. Create the next frontier

```
next_frontier = []
```

This will store the nodes discovered during the current hop.

For example:

Hop 1:

Engine



Fuel System

Pressure Sensor

next_frontier =

[Fuel System, Pressure Sensor]

These will be explored in the next hop.

11. Process every returned relationship

for row in rows:

Each row represents one relationship found by SPARQL.

Example:

Engine → connected_to → Fuel System

12. Get source and target

src, tgt = str(row.src), str(row.tgt)

Extract the source and target nodes.

Example:

src = Engine

tgt = Fuel System

13. Get the predicate

pred = str(row.predLabel) if row.predLabel else str(row.pred).rsplit("/", 1)[-1]

First it tries to use the readable predicate label.

Example:

connected_to

If a label isn't available, it takes the last part of the predicate URI.

So it has a **fallback** for missing labels.

14. Create a unique edge key

key = (src, pred, tgt)

The relationship is represented as:

(source, predicate, target)

Example:

(Engine, connected_to, Fuel System)

15. Check for duplicate relationship

if key in seen:

 continue

If this relationship was already processed, skip it.

This prevents duplicate edges.

16. Mark the relationship as seen

seen.add(key)

Now the system remembers:

"I have already processed this relationship."

17. Store the edge information

edges.append({

Now it creates a dictionary containing all information about this relationship.

Source

"source": src,

Who is the source node?

Source label

"source_label": str(row.srcLabel) if row.srcLabel else NODE_LABEL.get(src, src),

Gets a readable name for the source.

If SPARQL doesn't provide the label, it uses NODE_LABEL.

Predicate

"predicate": pred,

The relationship type.

Example:

connected_to

Target

"target": tgt,

The destination node.

Target label

"target_label": str(row.tgtLabel) if row.tgtLabel else NODE_LABEL.get(tgt, tgt),

Gets the readable target name.

Description

"description": str(row.description),

Stores the relationship description.

Strength

"strength": int(row.strength),

Stores the relationship strength.

For example:

95

Link type

"link_type": str(row.linkType),

Stores the type of relationship.

Hop number

"hop": hop,

Records how far this relationship is from the original seed.

For example:

1 → direct neighbour

2 → neighbour's neighbour

3 → third level

Seed score

```
"seed_score": max(  
    visited.get(src, (0, 0))[1],  
    visited.get(tgt, (0, 0))[1]  
)
```

This carries the **relevance score of the original seed** into the discovered relationship.

It checks the source and target and takes the higher seed score.

This helps later ranking.

18. Check both nodes of the edge

for other in (src, tgt):

This looks at both:

source

target

Why?

Because traversal works in **both directions**.

19. Check if node is already visited

if other not in visited:

If we haven't seen this node before, process it.

This prevents repeatedly exploring the same node.

20. Store newly discovered node

```
visited[other] = (hop, edges[-1]["seed_score"])
```

It records:

node

hop number

seed relevance score

Example:

Fuel System → (1, 95)

21. Add it to next frontier

```
next_frontier.append(other)
```

This means:

"Explore this node during the next hop."

So:

Current hop:

Engine → Fuel System

Next frontier:

Fuel System

22. Stop if maximum edges reached

```
if len(edges) >= MAX_EDGES:
```

```
    break
```

If we've reached the maximum number of edges, stop processing more relationships.

23. Move to the next hop

frontier = next_frontier

This is very important.

The nodes discovered in the current hop become the starting nodes for the next hop.

Example:

Hop 1:

Engine

↓

Fuel System

frontier = [Fuel System]

Hop 2:

Fuel System

↓

Fuel Pump

frontier = [Fuel Pump]

This is how multi-hop traversal happens.

24. Return the results

return edges, visited

Finally, the function returns two things:

edges

All relationships discovered.

Example:

Engine → connected_to → Fuel System

Fuel System → supplies → Fuel Pump

visited

All nodes discovered and their hop/seed information.

25. Final print

```
print("Defined: link_entities, lexical_fallback, traverse")
```

This simply prints a message confirming that the three functions have been created.

It does **not perform traversal**.

The entire traverse() flow

The easiest way to remember it:

Seed Nodes



Create Frontier



Run SPARQL



Find Neighbours



Store Edges



Find New Nodes



New Nodes become Next Frontier



Run SPARQL Again



Repeat until MAX_HOPS / MAX_EDGES



Return edges + visited nodes

In one sentence:

traverse() starts from the matched KG seed nodes, uses SPARQL to find their neighbours hop by hop, avoids duplicate nodes and relationships, records useful relationship information, and returns the collected graph evidence for the next ranking stage.

from the where it come the seed node meas how it define def traverse(seeds)

Yes — this is an important point.

When you see:

def traverse(seeds):

seeds is not created inside traverse().

It is **passed into the function from the previous step.**

Where do the seeds come from?

Earlier, you have:

link_entities(phrases)

This function finds KG nodes that match the phrases extracted by **Cell 15**.

For example:

User question

↓

Cell 15

↓

"hydraulic control system"

↓

link_entities()

↓

KG node:

Hydraulic_Control_System

↓

seed

link_entities() returns results like:

```
[
    (node1, "Hydraulic Control System", 95, "hydraulic control system"),
    (node2, "Hydraulic System", 88, "hydraulic control system")
]
```

Then somewhere later, the code calls:

```
traverse(seeds)
```

So seeds becomes that list.

What does this line do?

```
def traverse(seeds):
```

It simply means:

"Create a function called traverse that expects a variable called seeds."

It does **not** mean that seeds is defined there.

Then this line uses the seeds

```
visited = {n: (0, s) for n, _l, s, _p in seeds}
```

It takes each seed and extracts:

```
n = node
```

```
_l = label
```

```
s = score
```

```
_p = phrase
```

And:

```
frontier = [n for n, _l, _s, _p in seeds]
```

takes only the **node** from each seed.

So if the seeds are:

```
[
    (EngineNode, "Engine", 95, "engine"),
    (FuelNode, "Fuel System", 90, "fuel system")
]
```

]

then:

frontier

↓

[EngineNode, FuelNode]

Those are the nodes from which traversal starts.

Very simple:

Cell 15

↓

Extract phrases

↓

link_entities()

↓

Find matching KG nodes

↓

These matching nodes = SEEDS

↓

traverse(seeds)

↓

Explore neighbours

So **seeds are basically the starting points of graph traversal**, and they come from the entity-linking step, not from inside traverse().

where was the seed in the pervious step

The **seed is created in the previous step when link_entities() returns the matched KG nodes.**

In the code you shared, this line is important:

```
return [(n, NODE_LABEL.get(n, n), s, p)
```

```
        for n, (s, p) in sorted(best.items(), key=lambda kv: kv[1][0], reverse=True)]
```

Here:

- `n = KG node`
- `NODE_LABEL.get(n, n) = node label`
- `s = matching score`
- `p = original extracted phrase`

So `link_entities()` returns something like:

`(node1, "Engine", 95, "engine")`

`(node2, "Fuel System", 90, "fuel system")`

These returned **KG nodes (node1, node2) become the seeds** when the caller passes them to:

`traverse(seeds)`

Important distinction

There is **no variable literally called seed inside `link_entities()`**.

The code calls them:

`n`

when `link_entities()` returns them.

Then the next function treats those nodes as:

`seeds`

So the flow is:

Cell 15

↓

Extract phrases

↓

`link_entities(phrases)`

↓

Matched KG nodes

↓

returned as results

↓

passed as `seeds`



traverse(seeds)

And then inside traverse():

frontier = [n for n, _l, _s, _p in seeds]

extracts the actual node URIs from those seed results.

So: link_entities() finds the seeds; traverse() uses the seeds.

